

Checkers Mini AI Adversarial Search

Project report | Student: replace with your name | Register number: replace before submission

1. Problem formulation

The task is to choose a legal move for one side of an 8 by 8 English draughts board. A state is a board configuration plus the player whose turn it is. An action is a legal diagonal move or a complete capture sequence. The goal is to force the opponent into a state with no legal moves or no pieces. The agent treats Black as the maximizing player and Red as the minimizing player.

The essential constraint is compulsory capture. If any piece can jump an opponent, every quiet move is illegal. If that same piece can jump again after landing, the full chain must be completed in the same turn. Men move forward; kings move in both directions. A man that reaches the far row is crowned and its turn ends, following the English draughts convention.

2. Approach

The program uses depth-limited minimax with alpha-beta pruning. At each node it calls one shared `legal_moves(board, side)` function. This function first collects all capture chains for every piece; it returns those chains when any exist, otherwise it returns quiet diagonal moves. Consequently, the search tree itself contains only legal continuations and forced captures are respected at every maximizing and minimizing node.

A recursive capture generator removes the jumped piece, moves the capturing piece, and continues from its landing square. It emits a move only when no further jump is available, so a sequence such as 9x18x27 is one action rather than two optional actions. Move ordering tries captures and longer chains first to help alpha-beta prune earlier.

The evaluation is from the AI's perspective. It combines material (100 per man), king value (an additional 75), a small promotion-progress bonus for men, centre occupancy, and mobility. A terminal no-move state receives a score of plus or minus 100,000, which dominates heuristic values.

3. Complexity

With branching factor b and search depth d , plain minimax is $O(b^d)$ time and $O(d)$ recursion space, excluding stored boards. Alpha-beta has the same worst case but approaches $O(b^{d/2})$ with excellent ordering. Checkers branching is position-dependent: forced captures often reduce b sharply, while multi-jumps make one action longer to generate. The demonstration defaults to depth 5 to stay responsive without a transposition table.

4. Results

Check	Expected result	Observed result
Initial board	Seven Black opening moves	Pass
Forced capture	Quiet moves excluded when a jump exists	Pass
Multi-jump	9x18x27 generated as one action	Pass
Search legality	Alpha-beta chooses 9x18 in forced position	Pass
Evaluation	Extra own piece increases score	Pass

5. Reflection

The difficult part was representing a turn as an entire capture chain while still keeping the rules simple for search. Using one legal-move generator prevented the UI and agent from disagreeing about compulsory captures. A next improvement would add iterative deepening, transposition tables, and stronger positional features. Honesty note: AI assistance was used to help draft and implement this project; the submitted student should understand and be able to explain the design.

6. Demonstration evidence

The automated tests are run with `python -m unittest discover -s tests -v`. The command reports five passing tests. The command-line demonstration is run with `python -m src.game --depth 5`. It prints the board and the selected move after each turn. When a capture becomes available, the displayed legal move is written with x notation rather than a quiet move with a hyphen.

For the key forced-capture test, Black has a jump from square 9 over Red's piece on square 14 to square 18, while a different Black piece also has a quiet move. The legal-move list contains only 9x18, and the alpha-beta agent selects that capture. In the multi-jump test, Black's only action is 9x18x27; both jumped Red pieces disappear after applying the single returned move.

7. Limitations

This is an educational mini-agent rather than a tournament engine. It has no opening book, endgame database, draw-by-repetition detection, or transposition table. The evaluation is deliberately interpretable instead of learned. These choices keep the implementation compact while making the adversarial-search and forced-capture behaviour easy to inspect and defend.